

Chap 2 : GESTION DES PROCESSUS

2.1 Introduction

Un processus est un concept clé dans tout système d'exploitation et peut être schématisé comme un programme en cours d'exécution

Un processus contient toute l'information qu'il est nécessaire de conserver pour poursuivre l'exécution d'un programme interrompu.

Le SE est responsable de:

- Allocation de ressources aux processus
- Création, terminaison des processus
- Suspension, reprise des processus
- Synchronisation, communication entre processus.

2.2 Table des processus

La table des processus est un tableau de structures contenant toute l'information sur chaque processus en cours d'exécution sur le système.

Quelques champs typiques de la table des processus sur UNIX:

Gestion des processus	Gestion de la mémoire	Gestion des fichiers
Registres, compteur ordinal Mot d'état du programme Pointeur de pile État du processus Date de lancement du processus Temps CPU utilisé Temps CPU des fils Date de la prochaine alarme Pointeurs sur files de message Bits des signaux en attente Identificateur de processus (pid) Divers indicateurs	Pointeur sur le segment de code Pointeur sur segment de données Pointeur sur segment BSS Statut de fin d'exécution Statut de signal Identificateur de processus (pid) Processus père Groupe du processus uid réel, uid effectif gid réel, gid effectif Table de bits des signaux Divers indicateurs	Masque UMASK Répertoire racine Répertoire de travail Descripteurs de fichiers uid effectif gid effectif Paramètres des appels systèmes Divers indicateurs

Périodiquement, le système d'exploitation décide d'interrompre un processus pour en exécuter un autre.

Il doit alors:

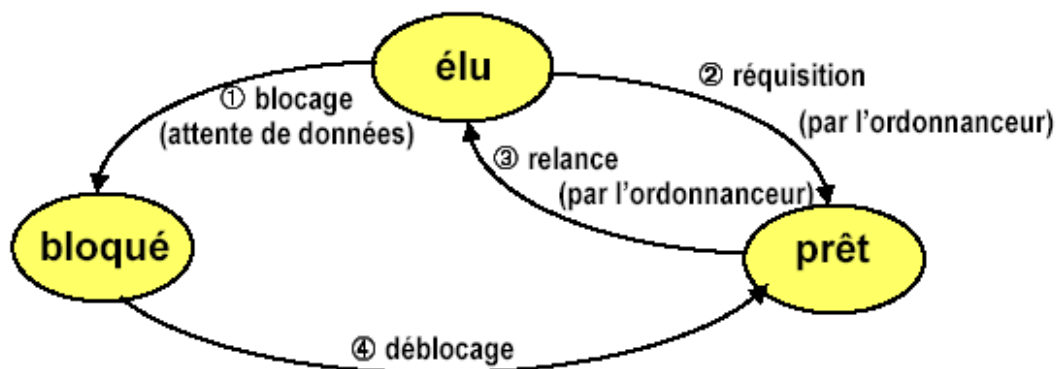
- Suspendre l'exécution d'un processus A
- Sauvegarder l'information de A dans la table des processus

- Choisir un nouveau processus B
- Récupérer l'information du processus B
- Démarrer l'exécution du processus B

2.3 Etats d'un Processus

Un processus peut connaître 3 états possibles :

- Élu (Actif) - Processus actif, Processeur en marche
- Prêt (Suspendu) – processus actif, processeur occupé par un autre processus
- Bloqué (En attente d'un évènement extérieur). Processus non actif, même si le processeur est disponible.



2.4 Ordonnanceur (Scheduler)

Partie du SE qui détermine si un processus doit poursuivre son exécution ou être temporairement arrêté pour être remplacé par un autre

- Le système d'exploitation est chargé de gérer l'allocation du processeur entre les différents programmes grâce à un algorithme d'ordonnancement.
- Le type d'ordonnanceur est totalement dépendant du système d'exploitation, en fonction de l'objectif visé.
- Doit choisir le prochain processus à être exécuté ce qui revient à gérer des Priorités.

Algorithme d'ordonnancement

- Un bon d'ordonnancement doit :
 - ✓ S'assurer que chaque processus reçoit sa part de temps CPU
 - ✓ Utiliser le CPU à 100%
 - ✓ Minimiser le temps de réponse pour les utilisateurs en mode interactif

- ✓ Minimiser l'attente des utilisateurs qui travaillent en traitement par lots
- ✓ Maximiser le nombre de travaux effectués en une heure

• Le système doit faire un choix selon les critères suivants :

- ✓ Équité : Chaque processus doit avoir du temps processeur.
- ✓ Efficacité : Le processeur doit être utilisé à 100 %
- ✓ Temps de réponse : L'utilisateur devant sa machine ne doit pas attendre.
- ✓ Temps d'exécution : Une séquence d'instructions ne doit pas trop durer.
- ✓ Rendement : Il faut faire le plus de choses possibles dans le temps le plus court possible.

Pour assurer qu'aucun processus ne s'exécute pendant trop de temps, pratiquement tous les ordinateurs ont un compteur de temps (timer) ou une horloge qui génère périodiquement une interruption (un signal).

A chaque interruption le système reprend la main et décide si le processus courant doit poursuivre son exécution ou s'il a épuisé le temps CPU qui lui était imparti, le processeur est alors alloué à un autre processus.

• Cette stratégie qui permet de suspendre des processus prêts est appelée ordonnancement avec réquisition (preemptive scheduling) et s'oppose à la méthode d'exécution jusqu'à achèvement (ordonnancement sans réquisition) des premiers systèmes à traitements par lots.

La préemption est la mise en attente forcée d'un processus :

- ✓ C'est l'ordonnanceur qui se charge de la gestion de la préemption
- ✓ Interruption d'horloges pour permettre la préemption par le SE;

• Les algorithmes d'ordonnancement sans réquisition sont simples et faciles à mettre en œuvre mais inadaptés à des systèmes généraux à plusieurs utilisateurs.

• Ordonnancement sans réquisition :

– Un processus est exécuté jusqu'à la fin.

• Ordonnancement avec réquisition :

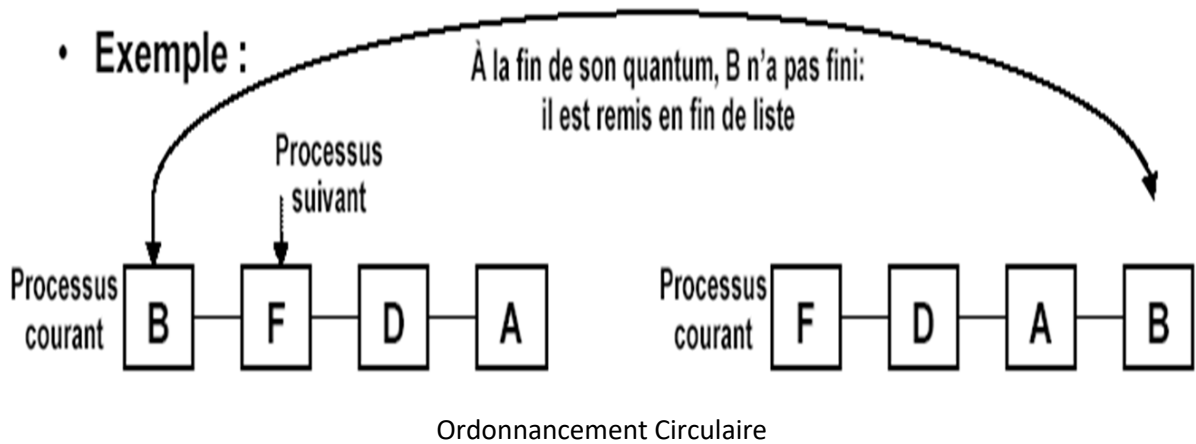
– A chaque signal d'horloge, le SE reprend la main, décide si le processus en cours a consommé son quota de temps et alloue éventuellement le processeur à un autre processus.

- Il existe de nombreux algorithmes d'ordonnancement.

2.4.1 Ordonnancement Circulaire (Tourniquet)

Chaque processus possède un quantum d'exécution:

- Si le processus a fini dans cet intervalle, on passe au suivant.
- S’il n’a pas fini, il passe en fin de liste et l’on exécute le suivant.



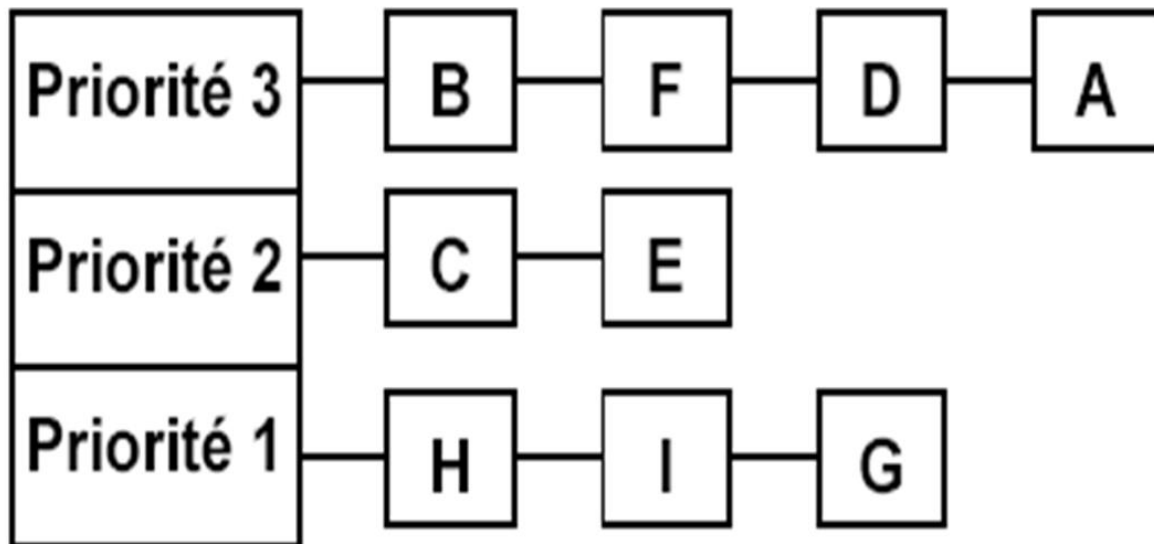
- ✓ Un des algos les plus anciens, les plus simples, les plus fiables et les plus utilisés (tourniquet ou round robin)
- ✓ Chaque processus possède un intervalle de temps, son quantum, durant lequel il est autorisé à s'exécuter
- ✓ Si le processus se bloque ou se termine avant la fin de son quantum, le CPU est immédiatement alloué à un autre processus
- ✓ L'ordonnanceur doit mémoriser une liste de processus prêts, lorsque que le quantum est épuisé le processus est mis en fin de liste

Problème : Réglage du Quantum :

- Quantum trop petit / Commutation : Le processeur passe son temps à commuter.
- Quantum trop grand : Augmentation du temps de réponse d'une commande

2.4.2 Ordonnancement avec priorité

- Plusieurs files d'attentes avec un niveau de priorité différent
- La priorité d'un processus décroît au cours du temps pour ne pas bloquer les autres files



2.4.3 Autres algorithmes

- Ordonnancement « plus court d'abord » :
 - Estimation de la durée de chaque processus en attente
 - Exécuter le processus le plus court
- Ordonnancement garanti :
 - Si n utilisateurs connectés, chacun reçoit $1/n$ du temps processeur.

2.5 Hiérarchie entre les processus

Les SE qui font appel au concept de processus doivent permettre de créer et détruire dynamiquement les processus.

Lorsqu'un processus en crée un autre, le père et l'enfant continuent d'être associés

- ✓ L'enfant peut lui-même créer d'autres processus
- ✓ Formation d'une hiérarchie de processus (init)
- ✓ Un processus et l'ensemble de ses descendants sont appelés un groupe de processus.

2.5.1 Informations sur un Processus

- Identificateur du processus (PID)
 - numéro unique (à un instant t, mais réutilisable)
- Informations de généalogie
 - processus parent (PPID)

- processus enfants
- Information de droits
- utilisateur propriétaire du processus
- utilisateur effectif (augmentation ou diminution de droits)
- Information sur la mémoire utilisée
- Information sur le temps passé
- Information sur les fichiers ouverts (ressources)

2.5.2 Thread

Afin de séparer les tâches d'un processus, il a été mis en place la notion de processus léger.

Chaque processus peut fonctionner comme un SE en lançant des sous tâches internes au processus.

On l'appelle le multithreading

- Ces sous-tâches sont nommées processus léger ou thread.
- Un processus peut comporter plusieurs threads.

2.6 Process Control Block (PCB)

- Structure de données du SE contenant les informations sur un processus.
- Ces informations sont dans une zone mémoire accessible uniquement par le noyau du SE.
- Pour obtenir des informations sur les processus, il est donc nécessaire de passer par des appels systèmes sous UNIX: • getpid, getppid, getuid, geteuid, setuid...